

UNIVERSIDAD DE COSTA RICA
ESCUELA DE INGENIERÍA ELÉCTRICA
IE0624 - LABORATORIO DE
MICROCONTROLADORES

INFORME DEL LABORATORIO 1
INTRODUCCIÓN A MICROCONTROLADORES Y
MANEJO DE GPIOS

PROFESOR
MARCO VILLALTA FALLAS

JOSE DANIEL BLANCO SOLIS, B71137

I 2025

25 DE MARZO DE 2025

Índice

1. Introducción	3
2. Nota teórica	4
2.1. Microcontrolador PIC12F683	4
2.2. Configuración de periféricos y registros utilizados	5
2.3. Generación de números pseudoaleatorios	5
2.4. Diseño del circuito	5
3. Desarrollo/Análisis	7
3.1. Diagrama de flujo	7
3.2. Código	8
3.3. Funcionalidad	8
4. Conclusiones y Recomendaciones	11
5. Anexos	13

Resumen

Este proyecto describe la implementación de un sistema basado en el microcontrolador PIC12F683, un botón y una matriz de LEDs, el cual se diseñó para mostrar números aleatorios utilizando un registro de desplazamiento con retroalimentación lineal, cuando se presiona el botón, el microcontrolador genera un número aleatorio entre 1 y 6 y enciende el conjunto correspondiente de LEDs para representar el número.

1. Introducción

El objetivo de este laboratorio fue desarrollar un sistema de dado digital utilizando el microcontrolador PIC12F683, varios LEDs para la visualización de números y un botón como entrada para simular el lanzamiento del dado, con esta práctica, se introdujo el manejo del simulador, los pines GPIO y la metodología de desarrollo en el curso, se implementó un algoritmo basado en un registro de desplazamiento de retroalimentación lineal para la generación de números pseudoaleatorios, obteniendo un sistema capaz de mostrar un número aleatorio entre 1 y 6 con el conjunto de LEDs al presionar el botón y apagar los LEDs después de un breve retraso.

2. Nota teórica

2.1. Microcontrolador PIC12F683

El PIC12F683 cuyo diagrama de bloques se observa en la figura 1, es un microcontrolador de 8 *bits* de la familia PIC de Microchip, conocido por su simplicidad y versatilidad, ofrece una memoria flash de 1,75 *KB*, 128 *bytes* de EEPROM y 128 *bytes* de RAM, su arquitectura Harvard permite una operación eficiente al separar los buses de instrucción y datos. Además, cuenta con un oscilador interno de 4 *MHz*, lo que simplifica el diseño al no requerir un cristal externo, entre sus características eléctricas opera con un voltaje de alimentación de 2,0 *V* a 5,5 *V*, con un consumo de corriente en operación que varía dependiendo del modo de trabajo y la frecuencia, en operación normal a 4 *MHz* el consumo típico es de aproximadamente 1,8 *mA* con una corriente de 50 *nA* en modo de bajo consumo, otra característica relevante es que posee seis pines GPIO, los cuales pueden configurarse de manera independiente como entradas o salidas mediante el registro TRISIO y algunos de estos pueden operar con funciones analógicas habilitadas o deshabilitadas mediante el registro ANSEL. [1].

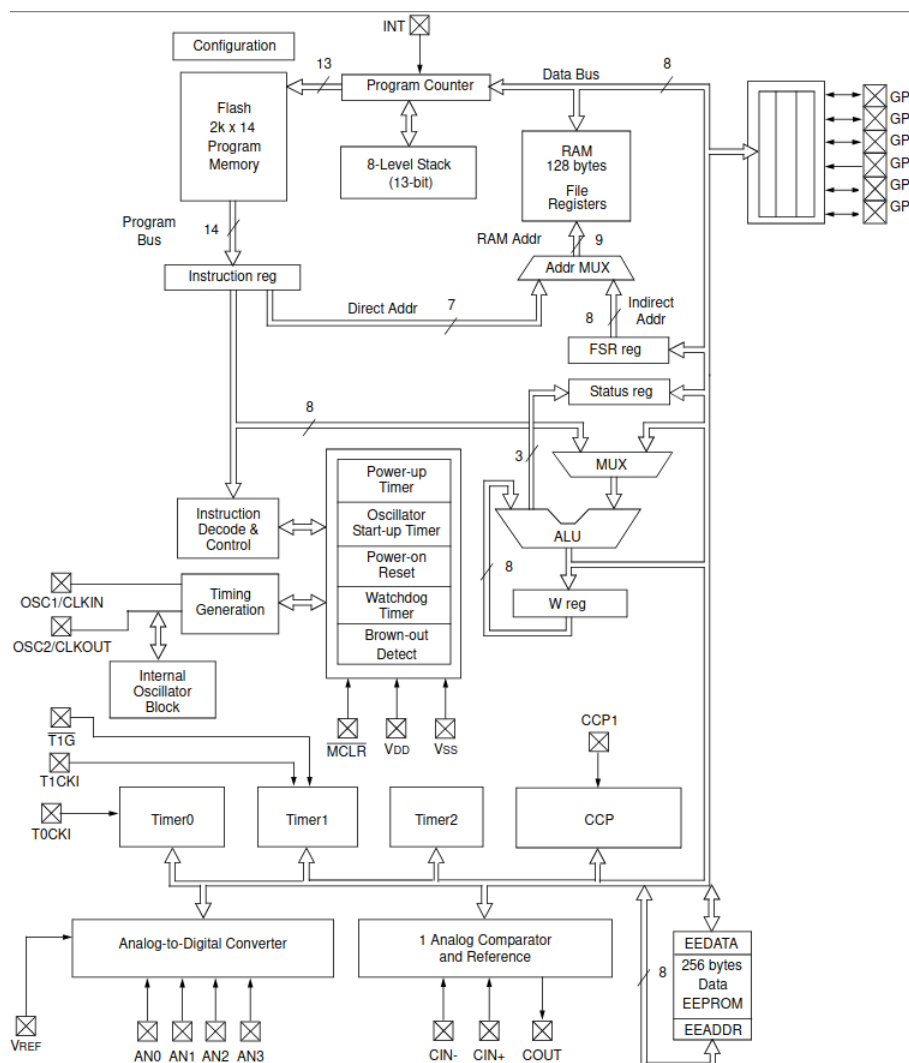


Figura 1: Diagrama de bloques del PIC12F683.

2.2. Configuración de periféricos y registros utilizados

Para evitar reinicios no deseados durante el desarrollo y prueba del sistema se desactivaron el Watchdog Timer (WDT) que es un temporizador que reinicia el microcontrolador si este deja de responder y la función de Master Clear Reset (MCLR) que proporciona una opción de reinicio manual, esto mediante la configuración del registro CONFIG, como se muestra en el siguiente bloque de código:

```
1 typedef unsigned int word;  
2 word __at 0x2007 __CONFIG = (_WDTE_OFF & _MCLRE_OFF);
```

El registro TRISIO se configuró de manera que GPIO4 funcione como entrada para el botón, mientras que los demás pines actúan como salidas para los LEDs, se estableció ANSEL = 0x00 para desactivar las funciones analógicas y operar exclusivamente en modo digital, lo cual es necesario para que los LEDs y el botón funcionen correctamente sin interferencias, además, se inicializaron todos los pines en estado bajo para evitar estados indeterminados al inicio.

2.3. Generación de números pseudoaleatorios

La generación de números pseudoaleatorios se implementó utilizando un registro de desplazamiento de retroalimentación lineal (LFSR), dado que se consideró el método más eficiente. El LFSR opera sobre 16 bits y utiliza operaciones de desplazamiento y XOR para producir nuevas secuencias cada vez que se presiona el botón [2], en este caso el resultado se limita para el rango entre 1 y 6.

2.4. Diseño del circuito

El circuito incluye un botón conectado a GPIO4 para proporcionar la entrada de usuario y 7 LEDs conectados a los pines GPIO para mostrar los números generados, el diseño contempla el uso de 7 LEDs, a pesar de que el sistema podría funcionar con 6 LEDs, el uso de un séptimo LED en el centro mejora la apariencia visual, dado que el costo de agregar un LED adicional es mínimo, esta decisión fue justificada para mejorar la calidad del proyecto. Debido a las limitaciones en la cantidad de pines GPIO disponibles, se optó por una configuración de LEDs en la que el central está conectado a un puerto, mientras que los de los bordes están agrupados de manera que dos LEDs comparten la misma conexión, lo que permite una mejor distribución de la luz y una apariencia más convincente de los números representados, permitiendo mostrar los números del 1 al 6 al activar combinaciones específicas de pines GPIO.

Para proteger tanto los LEDs como el microcontrolador, se integraron resistencias en serie con cada LED, estas limitan la corriente que pasa a través de los mismos asegurando que no exceda los límites máximos de corriente del microcontrolador que es de 25 mA por pin. Por otra parte, el rebote mecánico del botón puede generar pulsaciones erróneas debido a las transiciones rápidas entre estados, para evitar esto se agregó una solución simple pero efectiva para garantizar que el botón solo registre una pulsación por evento y es un retardo en el código tras la detección de una pulsación [3], además, se utilizó una resistencia en la conexión del botón para limitar la corriente cuando este se activa, evitando daños potenciales al microcontrolador.

El valor de la resistencia se calculó para garantizar que la corriente no supere los 25 mA cuando se utiliza una fuente de 5 V . Mediante la ecuación $V = IR$, se determinó que se necesita una resistencia de $200\ \Omega$. Para una mayor seguridad, esta resistencia fue dividida en dos, de manera que se distribuyera mejor la carga y se minimizara el riesgo de picos de corriente, el esquema del circuito se muestra en la siguiente imagen.

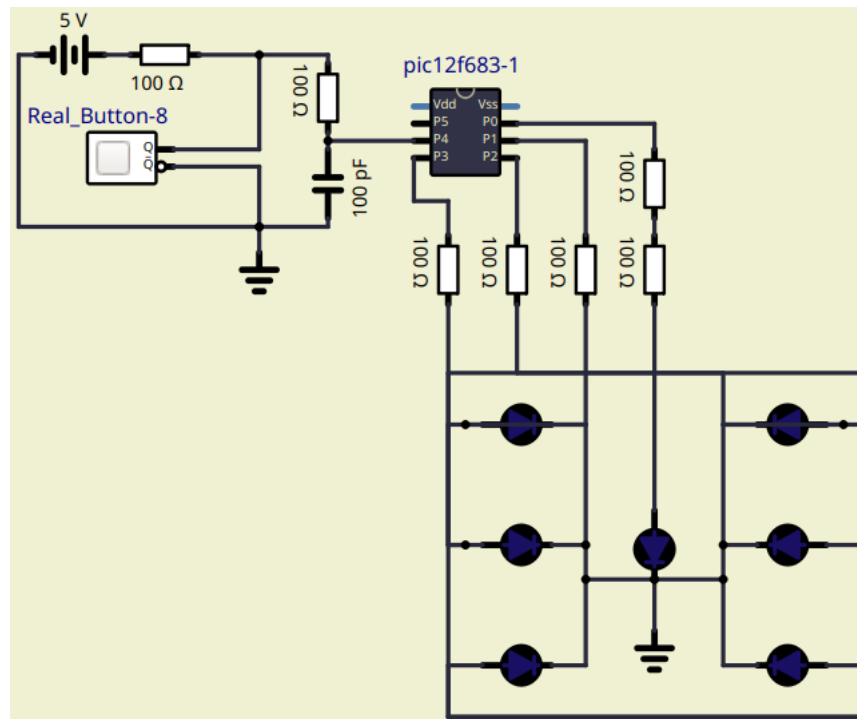


Figura 2: Circuito eléctrico.

La creación del circuito conlleva varios componentes los cuales se resumen en la siguiente tabla:

Tabla 1: Lista de componentes utilizados.

Cantidad	Componente	Descripción
1	PIC12F683	Microcontrolador de 8 bits
1	Botón pulsador	Entrada de usuario
7	LEDs	Indicadores visuales de números
7	Resistencias 100 Ω	Limitación de corriente para LEDs y Pull-down para el botón
1	Capacitor 100 nF	Pull-down para el botón

3. Desarrollo/Análisis

3.1. Diagrama de flujo

El flujo del programa incluye la inicialización del microcontrolador, donde se configuran los pines GPIO y se desactivan las funciones no esenciales, el sistema espera a que el botón conectado a GPIO4 sea presionado, cuando se detecta la pulsación se genera un número aleatorio mediante LFSR, ese número se traduce en una combinación específica de LEDs encendidos en los pines de salida representando el valor aleatorio obtenido, después de un retraso los LEDs se apagan y el programa vuelve a esperar una nueva entrada.

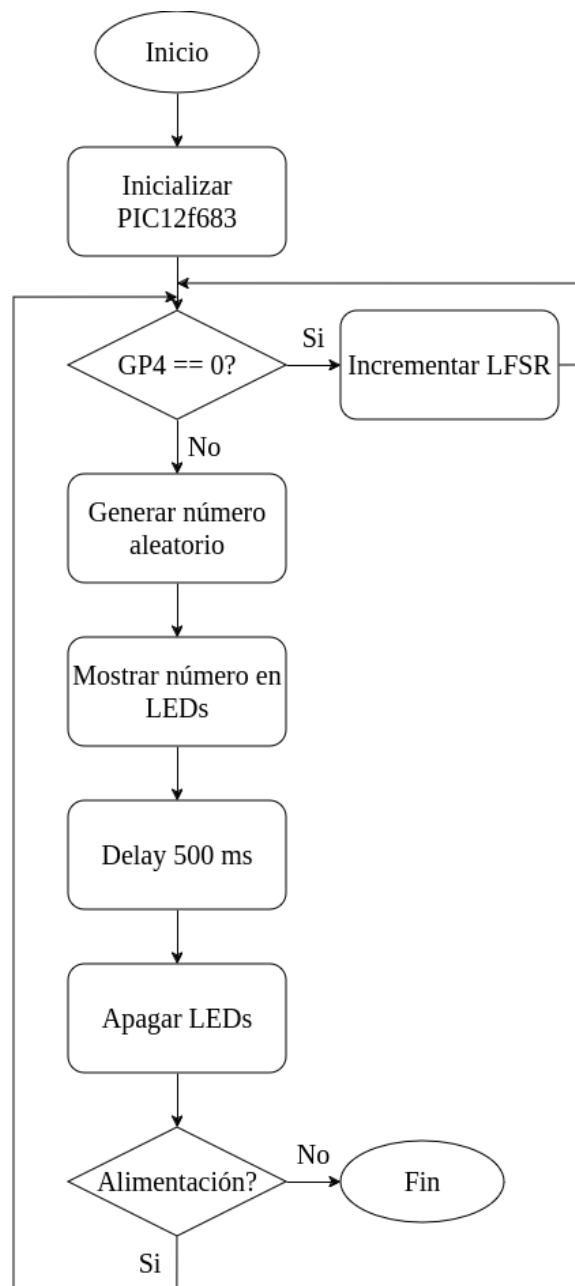


Figura 3: Diagrama de flujo del proceso.

3.2. Código

El código se estructuró con varias funciones, `initPIC12f683` configura el microcontrolador, asegurando que todos los pines estén en el estado adecuado antes de la operación, `generarNumeroAleatorio` usa un LFSR para generar números pseudoaleatorios, mientras que `mostrarNumero` controla los LEDs para mostrar el número generado, estas implementaciones se muestran a continuación:

```

1 void initPIC12f683() {
2     TRISIO = 0x10;
3     ANSEL = 0x00;
4     GPIO = 0x00;
5 }

1 uint8_t generarNumeroAleatorio() {
2     unsigned lsb = lfsr & 0x01; // Se obtiene el bit menos
        significativo
3     lfsr >>= 1; // Se desplaza el LFSR a la derecha
4     if (lsb) {
5         lfsr ^= 0xEF01u; // Se aplica XOR con la mscara si
            el bit menos significativo es 1
6     }
7     return (lfsr % 6) + 1; // Se retorna un valor entre 1 y 6
8 }

1 void mostrarNumero(int numero) {
2     GPIO &= 0b11100000;
3     switch (numero) {
4         case 1: GPIO |= 0b11100001; break;
5         case 2: GPIO |= 0b11100010; break;
6         case 3: GPIO |= 0b11100011; break;
7         case 4: GPIO |= 0b11100110; break;
8         case 5: GPIO |= 0b11100111; break;
9         case 6: GPIO |= 0b11101110; break;
10    }
11 }

```

3.3. Funcionalidad

El botón está conectado al pin GPIO4, configurado como entrada digital con una resistencia pull-down para asegurar que su estado en reposo sea 0, al presionar el botón se genera una señal de alto (1 lógico) que es detectada por el microcontrolador e inicia el proceso de generación del número aleatorio. Los LEDs están conectados a los pines GPIO0-GPIO3 y GPIO5, configurados como salidas, para representar cada número se encienden diferentes combinaciones de LEDs, la activación se controla mediante operaciones lógicas sobre el registro GPIO.

En términos de consumo de energía, el sistema presenta una eficiencia aceptable debido a la baja potencia requerida por el PIC12F683 y los LEDs, sin embargo, dado que el microcontrolador está

continuamente ejecutando el código en un bucle infinito podría optimizarse el consumo energético implementando modos de bajo consumo cuando el sistema está inactivo y “despertando” solo cuando se presiona el botón.

El análisis de la distribución de los valores generados mostró que el LFSR proporciona una secuencia aleatoria aceptablemente uniforme, se realizaron pruebas de conteo en las que se observó que tras múltiples iteraciones, los valores del 1 al 6 aparecían con una frecuencia similar lo que confirma la efectividad del método de generación pseudoaleatoria implementado en el código.

Para validar el funcionamiento del circuito, se realizaron diversas pruebas en las que se observó el comportamiento de los LEDs en respuesta a la pulsación del botón, el sistema respondió según lo esperado generando y mostrando números aleatorios cada vez que se presionaba el botón, los LEDs correspondientes se encendieron y apagaron después de un retraso proporcionando una visualización clara del número generado. A continuación, se presentan imágenes del sistema en operación:

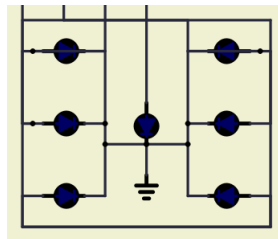


Figura 4: Estado inicial del sistema con los LEDs apagados.

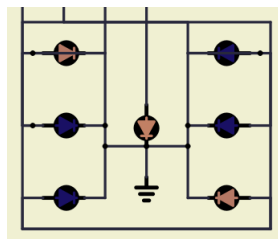


Figura 5: Se encienden LEDs correspondientes al valor aleatorio obtenido.

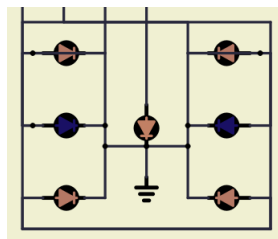


Figura 6: Se encienden LEDs correspondientes al valor aleatorio obtenido.

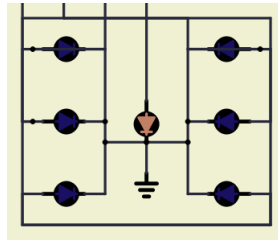


Figura 7: Se encienden LEDs correspondientes al valor aleatorio obtenido.

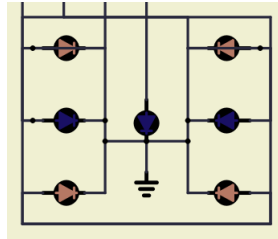


Figura 8: Se encienden LEDs correspondientes al valor aleatorio obtenido.

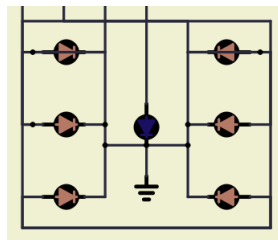


Figura 9: Se encienden LEDs correspondientes al valor aleatorio obtenido.

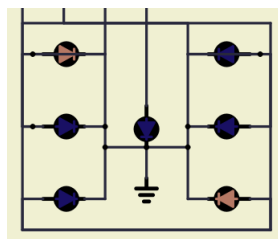


Figura 10: Se encienden LEDs correspondientes al valor aleatorio obtenido.

4. Conclusiones y Recomendaciones

La implementación del sistema presenta un diseño funcional con una correcta integración entre el hardware y el software, la utilización de LFSR para la generación de números aleatorios fue eficaz ante la falta de funciones definidas para este fin y la interacción con el usuario mediante el botón es intuitiva, para futuras mejoras se podría implementar un debounce más robusto utilizando un temporizador en lugar de un simple retardo, también se recomienda investigar técnicas de inicialización de la semilla para mejorar la aleatoriedad de los números generados o la optimización del consumo energético utilizando modos de bajo consumo en el microcontrolador. La implementación completa de este proyecto se puede acceder desde el siguiente repositorio [git](#).

Referencias

- [1] Microchip (2007) *PIC12F683*. Recuperado de: <https://www.microchip.com/en-us/product/PIC12F683>
- [2] Analog Devices (2010). *Pseudo Random Number Generation Using Linear Feedback Shift Registers*. Recuperado de: <https://www.analog.com/en/resources/design-notes/random-number-generation-using-lfsr.html>
- [3] Jack Ganssle (2014). *A Guide to Debouncing*. Recuperado de: <http://www.ganssle.com/debouncing.pdf>

5. Anexos



PIC12F683

Data Sheet

8-Pin Flash-Based, 8-Bit
CMOS Microcontrollers with
nanoWatt Technology

* 8-bit, 8-pin Devices Protected by Microchip's Low Pin Count Patent: U.S. Patent No. 5,847,450. Additional U.S. and foreign patents and applications may be issued or pending.

Note the following details of the code protection feature on Microchip devices:

- Microchip products meet the specification contained in their particular Microchip Data Sheet.
- Microchip believes that its family of products is one of the most secure families of its kind on the market today, when used in the intended manner and under normal conditions.
- There are dishonest and possibly illegal methods used to breach the code protection feature. All of these methods, to our knowledge, require using the Microchip products in a manner outside the operating specifications contained in Microchip's Data Sheets. Most likely, the person doing so is engaged in theft of intellectual property.
- Microchip is willing to work with the customer who is concerned about the integrity of their code.
- Neither Microchip nor any other semiconductor manufacturer can guarantee the security of their code. Code protection does not mean that we are guaranteeing the product as "unbreakable."

Code protection is constantly evolving. We at Microchip are committed to continuously improving the code protection features of our products. Attempts to break Microchip's code protection feature may be a violation of the Digital Millennium Copyright Act. If such acts allow unauthorized access to your software or other copyrighted work, you may have a right to sue for relief under that Act.

Information contained in this publication regarding device applications and the like is intended through suggestion only and may be superseded by updates. It is your responsibility to ensure that your application meets with your specifications. No representation or warranty is given and no liability is assumed by Microchip Technology Incorporated with respect to the accuracy or use of such information, or infringement of patents or other intellectual property rights arising from such use or otherwise. Use of Microchip's products as critical components in life support systems is not authorized except with express written approval by Microchip. No licenses are conveyed, implicitly or otherwise, under any intellectual property rights.

**QUALITY MANAGEMENT SYSTEM
CERTIFIED BY DNV
== ISO/TS 16949:2002 ==**

8-Pin Flash-Based, 8-Bit CMOS Microcontrollers with nanoWatt Technology

High-Performance RISC CPU

- Only 35 instructions to learn:
 - All single-cycle instructions except branches
- Operating speed:
 - DC – 20 MHz oscillator/clock input
 - DC – 200 ns instruction cycle
- Interrupt capability
- 8-level deep hardware stack
- Direct, Indirect and Relative Addressing modes

Special Microcontroller Features

- Precision Internal Oscillator:
 - Factory calibrated to $\pm 1\%$
 - Software selectable frequency range of 8 MHz to 31 kHz
 - Two-speed Start-up mode
 - Crystal fail detect for critical applications
 - Clock mode switching during operation for power savings
- Power-saving Sleep mode
- Wide operating voltage range. (2.0V-5.5V)
- Industrial and Extended temperature range
- Power-on Reset (POR)
- Power-up Timer (PWRT) and Oscillator Start-up Timer (OST)
- Multiplexed Master Clear with pull-up/input pin
- Programmable code protection
- High Endurance Flash/EEPROM cell:
 - 100,000 write Flash endurance
 - 1,000,000 write EEPROM endurance
 - Flash/Data EEPROM Retention: > 40 years

Low-Power Features

- Standby Current:
 - 1 nA @ 2.0V, typical
- Operating Current:
 - 8.5 μ A @ 32 kHz, 2.0V, typical
 - 100 μ A @ 1 MHz, 2.0V, typical
- Watchdog Timer Current:
 - 1 μ A @ 2.0V, typical

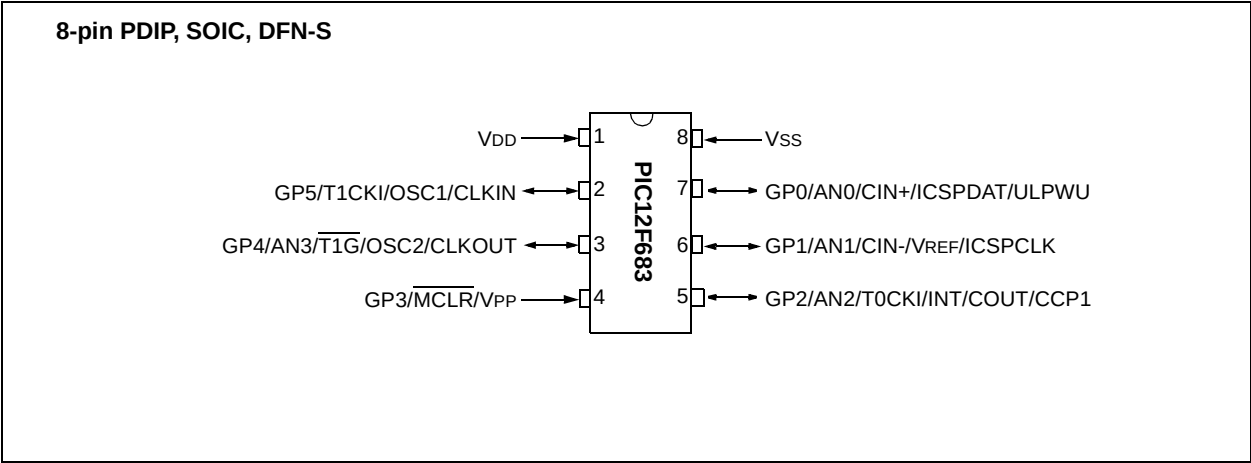
Peripheral Features

- 6 I/O pins with individual direction control:
 - High current source/sink for direct LED drive
 - Interrupt-on-pin change
 - Individually programmable weak pull-ups
 - Ultra Low-Power Wake-up on GP0
- Analog comparator module with:
 - One analog comparator
 - Programmable on-chip voltage reference (CVREF) module (% of VDD)
 - Comparator inputs and output externally accessible
- A/D Converter:
 - 10-bit resolution and 4 channels
- Timer0: 8-bit timer/counter with 8-bit programmable prescaler
- Enhanced Timer1:
 - 16-bit timer/counter with prescaler
 - External Gate Input mode
 - Option to use OSC1 and OSC2 in LP mode as Timer1 oscillator if INTOSC mode selected
- Timer2: 8-bit timer/counter with 8-bit period register, prescaler and postscaler
- Capture, Compare, PWM module:
 - 16-bit Capture, max resolution 12.5 ns
 - Compare, max resolution 200 ns
 - 10-bit PWM, max frequency 20 kHz
- In-Circuit Serial Programming™ (ICSP™) via two pins

Device	Program Memory	Data Memory		I/O	10-bit A/D (ch)	Comparators	Timers 8/16-bit
	Flash (words)	SRAM (bytes)	EEPROM (bytes)				
PIC12F683	2048	128	256	6	4	1	2/1

PIC12F683

Pin Diagram



4.0 GPIO PORT

There are as many as six general purpose I/O pins available. Depending on which peripherals are enabled, some or all of the pins may not be available as general purpose I/O. In general, when a peripheral is enabled, the associated pin may not be used as a general purpose I/O pin.

Note: Additional information on I/O ports may be found in the “*PICmicro® Mid-Range MCU Family Reference Manual*” (DS33023).

4.1 GPIO and the TRISIO Registers

GPIO is a 6-bit wide, bidirectional port. The corresponding data direction register is TRISIO. Setting a TRISIO bit (= 1) will make the corresponding GPIO pin an input (i.e., put the corresponding output driver in a High-impedance mode). Clearing a TRISIO bit (= 0) will make the corresponding GPIO pin an output (i.e., put the contents of the output latch on the selected pin). The exception is GP3, which is input only and its TRISIO bit will always read as ‘1’. Example 4-1 shows how to initialize GPIO.

Reading the GPIO register reads the status of the pins, whereas writing to it will write to the port latch. All write operations are read-modify-write operations. Therefore, a write to a port implies that the port pins are read, this value is modified and then written to the port data latch. GP3 reads ‘0’ when MCLR = 1.

The TRISIO register controls the direction of the GPIO pins, even when they are being used as analog inputs. The user must ensure the bits in the TRISIO register are maintained set when using them as analog inputs. I/O pins configured as analog input always read ‘0’.

Note: The ANSEL (9Fh) and CMCON0 (19h) registers must be initialized to configure an analog channel as a digital input. Pins configured as analog inputs will read ‘0’.

EXAMPLE 4-1: INITIALIZING GPIO

```
BCF    STATUS,RP0    ;Bank 0
CLRF   GPIO          ;Init GPIO
MOVLW  07h           ;Set GP<2:0> to
MOVWF  CMCON0        ;digital I/O
BSF    STATUS,RP0    ;Bank 1
CLRF   ANSEL         ;digital I/O
MOVLW  0Ch           ;Set GP<3:2> as inputs
MOVWF  TRISIO        ;and set GP<5:4,1:0>
                          ;as outputs
BCF    STATUS,RP0    ;Bank 0
```

4.2 Additional Pin Functions

Every GPIO pin on the PIC12F683 has an interrupt-on-change option and a weak pull-up option. GP0 has an Ultra Low-Power Wake-up option. The next three sections describe these functions.

4.2.1 WEAK PULL-UPS

Each of the GPIO pins, except GP3, has an individually configurable weak internal pull-up. Control bits WPUx enable or disable each pull-up. Refer to Register 4-3. Each weak pull-up is automatically turned off when the port pin is configured as an output. The pull-ups are disabled on a Power-on Reset by the GPPU bit (OPTION<7>). A weak pull-up is automatically enabled for GP3 when configured as MCLR and disabled when GP3 is an I/O. There is no software control of the MCLR pull-up.

REGISTER 4-1: GPIO – GENERAL PURPOSE I/O REGISTER (ADDRESS: 05h)

U-0	U-0	R/W-x	R/W-x	R/W-x	R/W-x	R/W-0	R/W-0
—	—	GP5	GP4	GP3	GP2	GP1	GP0
bit 7		bit 0					

bit 7-6: **Unimplemented:** Read as ‘0’

bit 5-0: **GPIO<5:0>:** GPIO I/O pin

1 = Port pin is > V_{IH}

0 = Port pin is < V_{IL}

Legend:

R = Readable bit

W = Writable bit

U = Unimplemented bit, read as ‘0’

- n = Value at POR

‘1’ = Bit is set

‘0’ = Bit is cleared

x = Bit is unknown

PIC12F683

REGISTER 4-2: TRISIO – GPIO TRI-STATE REGISTER (ADDRESS: 85h)

U-0	U-0	R/W-1	R/W-1	R-1	R/W-1	R/W-1	R/W-1	
—	—	TRISIO5	TRISIO4	TRISIO3	TRISIO2	TRISIO1	TRISIO0	
bit 7								bit 0

bit 7-6: **Unimplemented:** Read as '0'

bit 5-0: **TRISIO<5:0>**: GPIO Tri-State Control bit
 1 = GPIO pin configured as an input (tri-stated)
 0 = GPIO pin configured as an output

- Note 1:** TRISIO<3> always reads '1'.
2: TRISIO<5:4> reads '1' in XT, LP and HS modes.

Legend:

R = Readable bit W = Writable bit U = Unimplemented bit, read as '0'
 - n = Value at POR '1' = Bit is set '0' = Bit is cleared x = Bit is unknown

REGISTER 4-3: WPU – WEAK PULL-UP REGISTER (ADDRESS: 95h)

U-0	U-0	R/W-1	R/W-1	U-0	R/W-1	R/W-1	R/W-1	
—	—	WPU5	WPU4	—	WPU2	WPU1	WPU0	
bit 7								bit 0

bit 7-6: **Unimplemented:** Read as '0'

bit 5-4: **WPU<5:4>**: Weak Pull-up register bit
 1 = Pull-up enabled
 0 = Pull-up disabled

bit 3: **Unimplemented:** Read as '0'

bit 2-0: **WPU<2:0>**: Weak Pull-up register bit
 1 = Pull-up enabled
 0 = Pull-up disabled

- Note 1:** Global $\overline{\text{GPPU}}$ must be enabled for individual pull-ups to be enabled.
2: The weak pull-up device is automatically disabled if the pin is in output mode (TRISIO = 0).
3: The GP3 pull-up is enabled when configured as $\overline{\text{MCLR}}$ and disabled as an I/O in the Configuration Word.
4: WPU<5:4> reads '1' in XT, LP and HS modes.

Legend:

R = Readable bit W = Writable bit U = Unimplemented bit, read as '0'
 - n = Value at POR '1' = Bit is set '0' = Bit is cleared x = Bit is unknown

4.2.2 INTERRUPT-ON-CHANGE

Each of the GPIO pins is individually configurable as an interrupt-on-change pin. Control bits IOC_x enable or disable the interrupt function for each pin. Refer to Register 4-4. The interrupt-on-change is disabled on a Power-on Reset.

For enabled interrupt-on-change pins, the values are compared with the old value latched on the last read of GPIO. The 'mismatch' outputs of the last read are OR'd together to set the GPIO Change Interrupt Flag bit (GPIF) in the INTCON register.

This interrupt can wake the device from Sleep. The user, in the Interrupt Service Routine, clears the interrupt by:

- Any read or write of GPIO. This will end the mismatch condition, then
- Clear the flag bit GPIF.

A mismatch condition will continue to set flag bit GPIF. Reading GPIO will end the mismatch condition and allow flag bit GPIF to be cleared. The latch holding the last read value is not affected by a MCLR nor BOD Reset. After these resets, the GPIF flag will continue to be set if a mismatch is present.

Note: If a change on the I/O pin should occur when the read operation is being executed (start of the Q2 cycle), then the GPIF interrupt flag may not get set.

REGISTER 4-4: IOC – INTERRUPT-ON-CHANGE GPIO REGISTER (ADDRESS: 96h)

U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
—	—	IOC5	IOC4	IOC3	IOC2	IOC1	IOC0
bit 7		bit 0					

bit 7-6 **Unimplemented:** Read as '0'

bit 5-0 **IOC<5:0>:** Interrupt-on-change GPIO Control bit
 1 = Interrupt-on-change enabled
 0 = Interrupt-on-change disabled

Note 1: Global Interrupt Enable (GIE) must be enabled for individual interrupts to be recognized.

2: IOC<5:4> reads '1' in XT, LP and HS modes.

Legend:

R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'
- n = Value at POR	'1' = Bit is set	'0' = Bit is cleared x = Bit is unknown

4.2.3 ULTRA LOW-POWER WAKE-UP

The Ultra Low-Power Wake-up (ULPWU) on GP0 allows a slow falling voltage to generate an interrupt-on-change on GP0 without excess current consumption. The mode is selected by setting the ULPWUE bit (PCON<5>). This enables a small current sink which can be used to discharge a capacitor on GP0.

To use this feature, the GP0 pin is configured to output '1' to charge the capacitor, interrupt-on-change for GP0 is enabled and GP0 is configured as an input. The ULPWUE bit is set to begin the discharge and a SLEEP instruction is performed. When the voltage on GP0 drops below V_{IL}, an interrupt will be generated which will cause the device to wake-up. Depending on the state of the GIE bit (INTCON<7>), the device will either jump to the interrupt vector (0004h) or execute the next instruction when the interrupt event occurs. See **Section 4.2.2 "Interrupt-on-change"** and **Section 12.4.3 "GPIO Interrupt"** for more information.

This feature provides a low-power technique for periodically waking up the device from Sleep. The time-out is dependent on the discharge time of the RC circuit on GP0. See Example 4-2 for initializing the Ultra Low-Power Wake-up module.

The series resistor provides overcurrent protection for the GP0 pin and can allow for software calibration of the time-out (see Figure 4-1). A timer can be used to measure the charge time and discharge time of the capacitor. The charge time can then be adjusted to provide the desired interrupt delay. This technique will compensate for the affects of temperature, voltage and component accuracy. The Ultra Low-Power Wake-up peripheral can also be configured as a simple Programmable Low-Voltage Detect or temperature sensor.

Note: For more information, refer to the Application Note AN879, "Using the Microchip Ultra Low-Power Wake-up Module" (DS00879).

PIC12F683

EXAMPLE 4-2: ULTRA LOW-POWER WAKE-UP INITIALIZATION

```
BCF      STATUS,RP0      ;Bank 0
BSF      GPIO,0          ;Set GP0 data latch
MOVLW   H'7'            ;Turn off
MOVWF   CMCON0           ; comparator
BSF      STATUS,RP0      ;Bank 1
BCF      ANSEL,0         ;GP0 to digital I/O
BCF      TRISIO,0        ;Output high to
CALL    CapDelay         ; charge capacitor
BSF      PCON,ULPWUE     ;Enable ULP Wake-up
BSF      IOC,0           ;Select GP0 IOC
BSF      TRISIO,0        ;GP0 to input
MOVLW   B'10001000'     ;Enable interrupt
MOVWF   INTCON           ; and clear flag
SLEEP                   ;wait for IOC
```

4.2.4 PIN DESCRIPTIONS AND DIAGRAMS

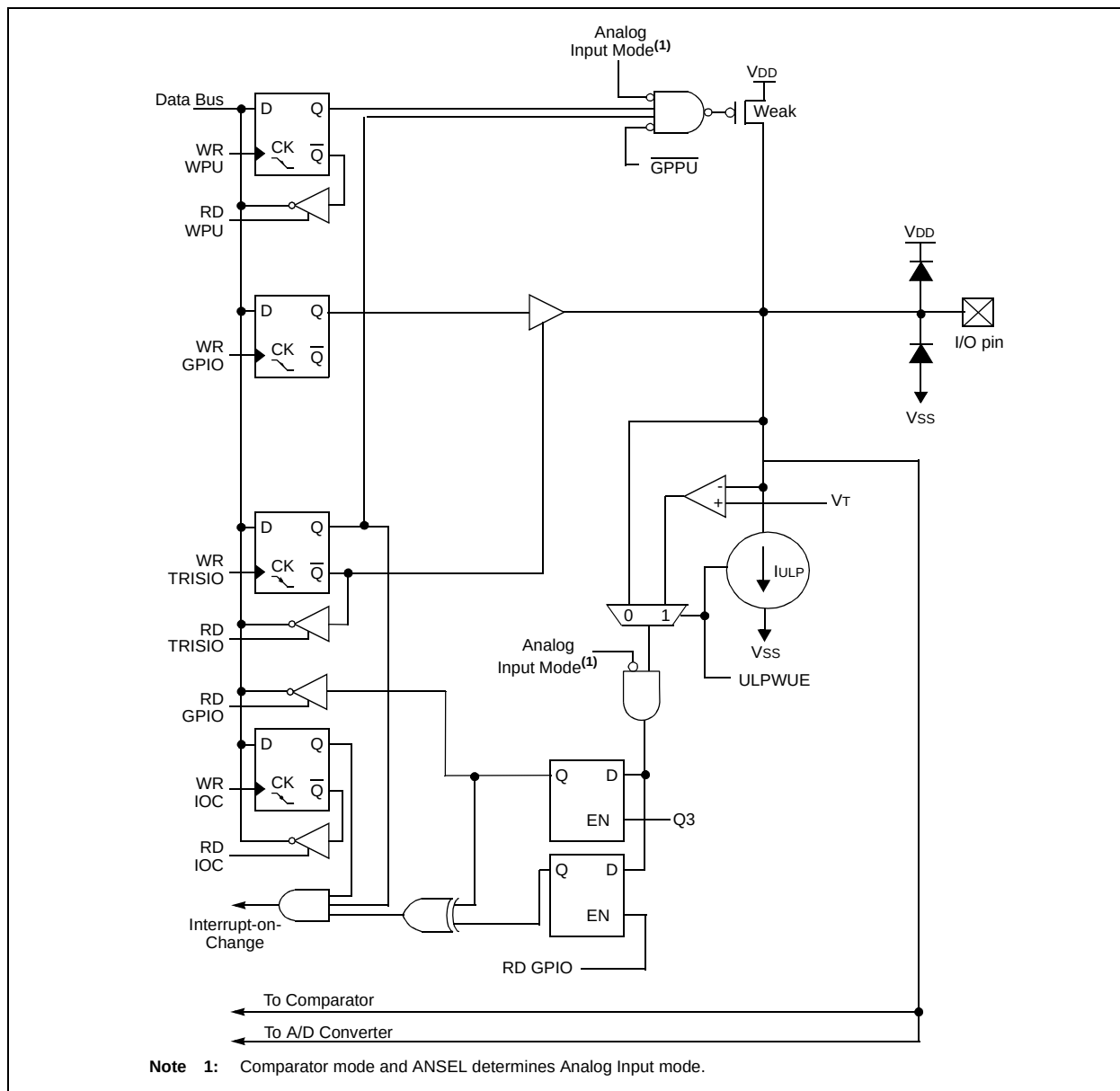
Each GPIO pin is multiplexed with other functions. The pins and their combined functions are briefly described here. For specific information about individual functions such as the comparator or the A/D, refer to the appropriate section in this data sheet.

4.2.4.1 GP0/AN0/CIN+/ICSPDAT/ULPWU

Figure 4-1 shows the diagram for this pin. The GP0 pin is configurable to function as one of the following:

- a general purpose I/O
- an analog input for the A/D
- an analog input to the comparator
- an analog input to the Ultra Low-Power Wake-up
- In-Circuit Serial Programming data

FIGURE 4-1: BLOCK DIAGRAM OF GP0

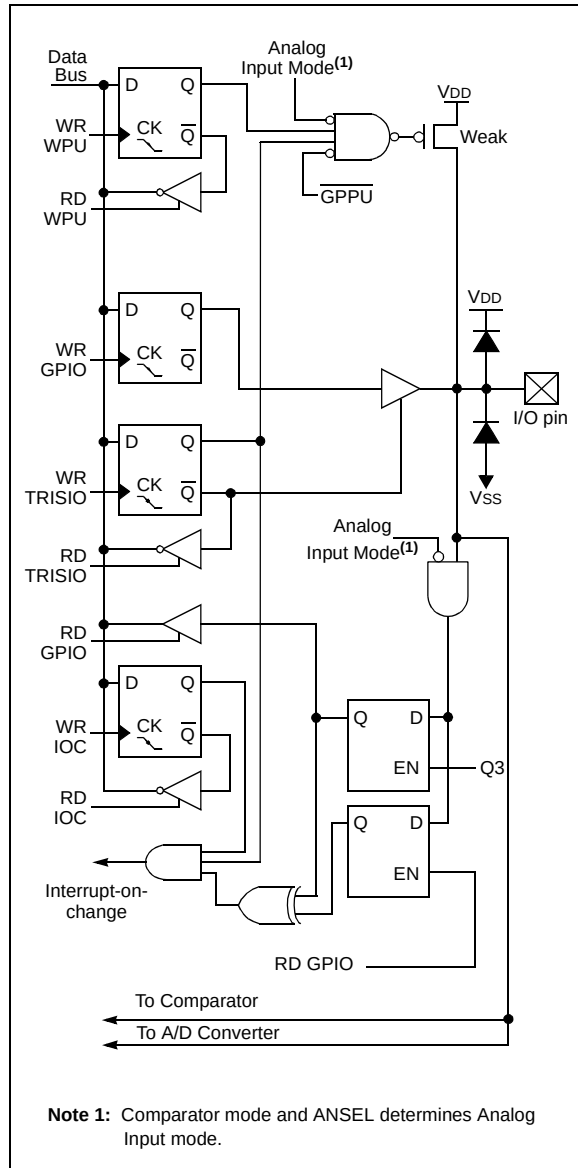


4.2.4.2 GP1/AN1/CIN-/VREF/ICSPCLK

Figure 4-1 shows the diagram for this pin. The GP1 pin is configurable to function as one of the following:

- a general purpose I/O
- an analog input for the A/D
- a analog input to the comparator
- a voltage reference input for the A/D
- In-Circuit Serial Programming clock

FIGURE 4-2: BLOCK DIAGRAM OF GP1

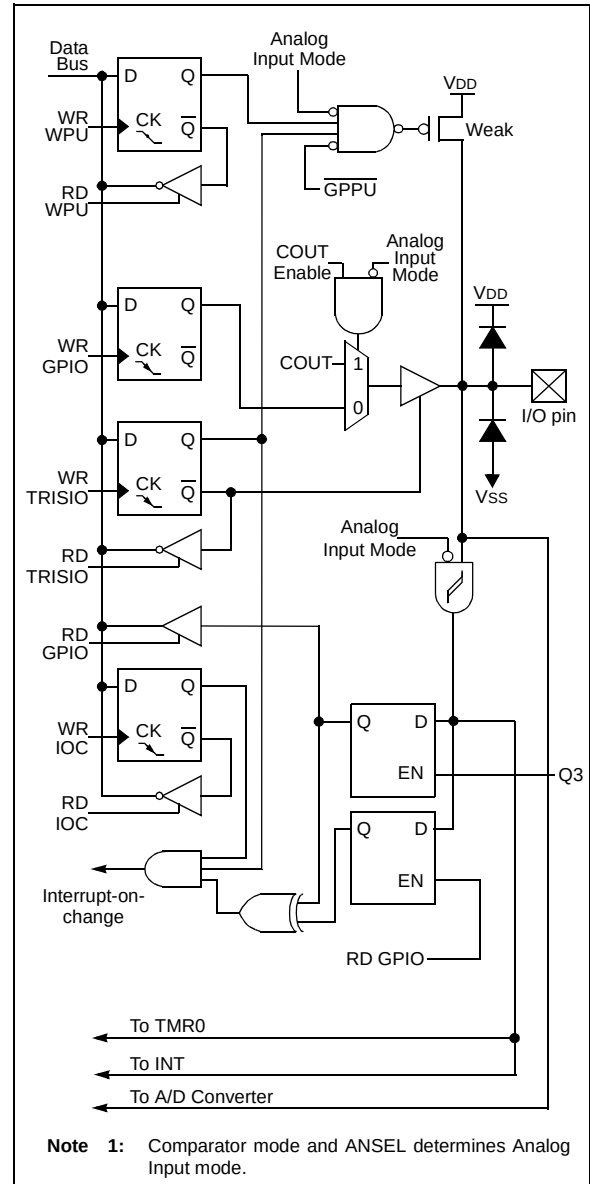


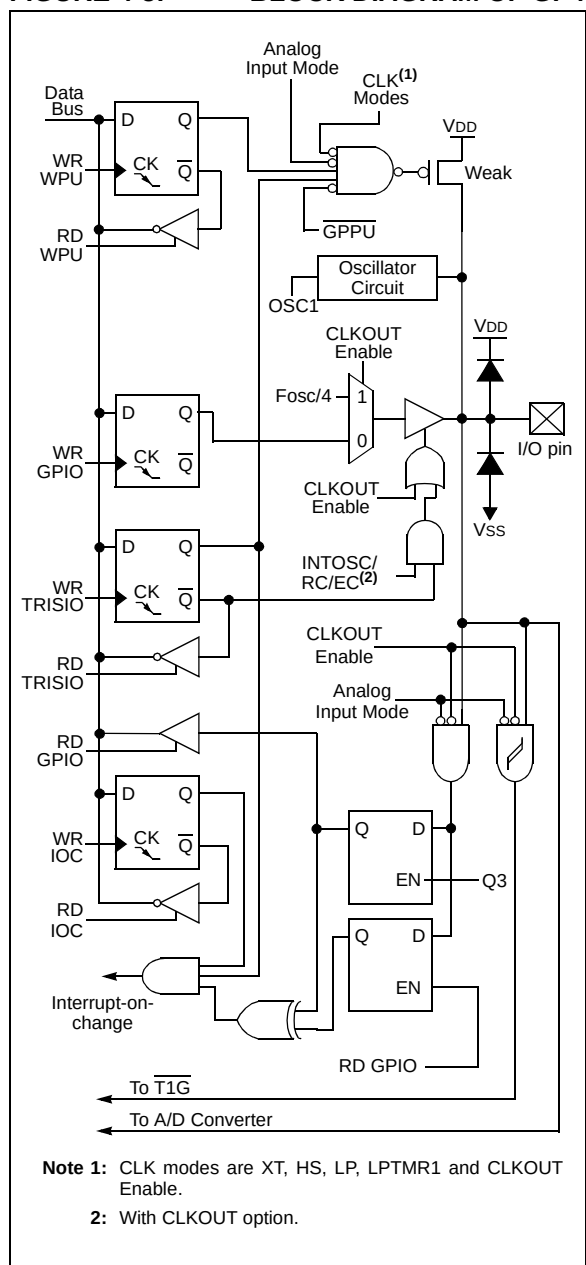
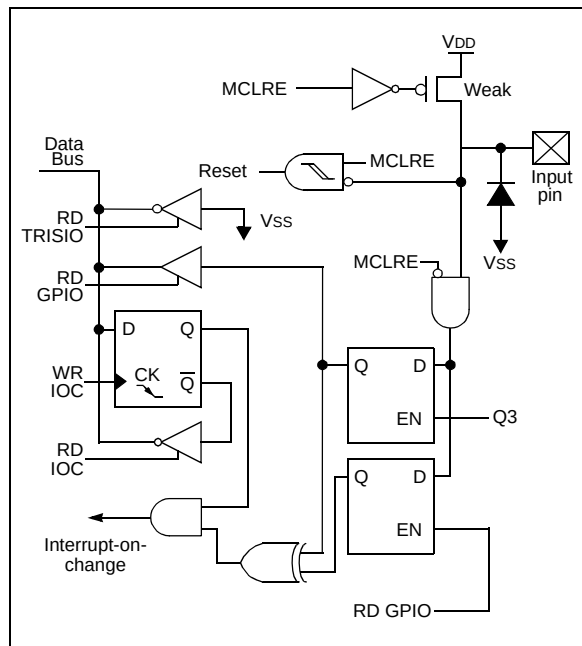
4.2.4.3 GP2/AN2/T0CKI/INT/COUT/CCP1

Figure 4-3 shows the diagram for this pin. The GP2 pin is configurable to function as one of the following:

- a general purpose I/O
- an analog input for the A/D
- the clock input for TMR0
- an external edge triggered interrupt
- a digital output from the comparator
- a digital input/output for the CCP (refer to Section 11.0 "Capture/Compare/PWM (CCP) Module").

FIGURE 4-3: BLOCK DIAGRAM OF GP2



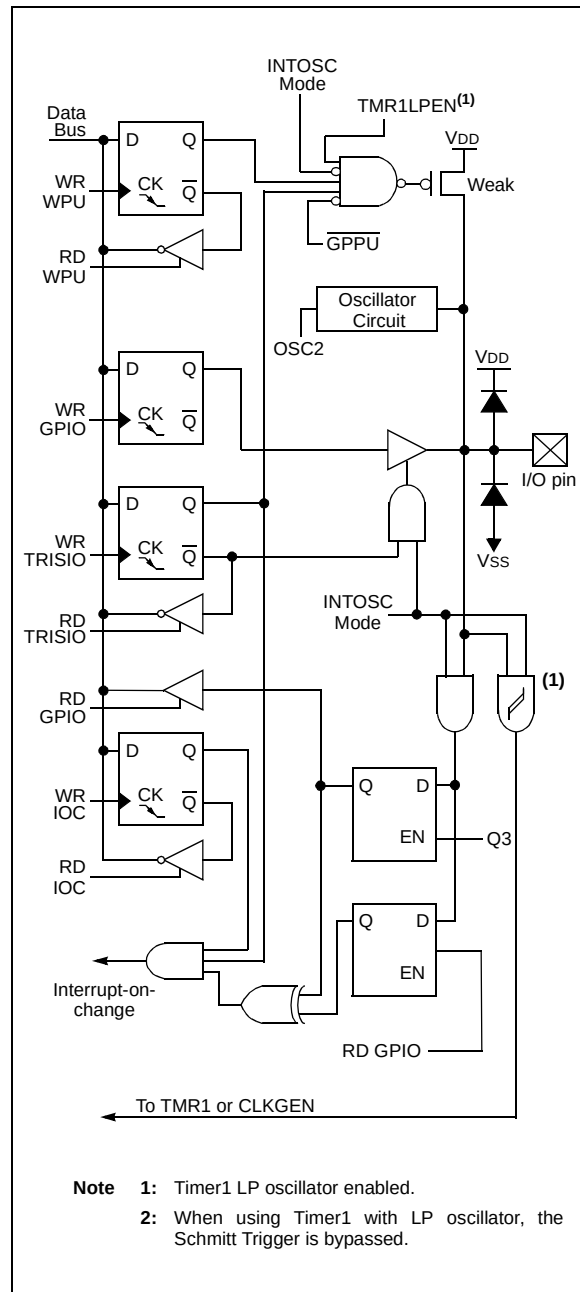


4.2.4.6 GP5/T1CKI/OSC1/CLKIN

Figure 4-6 shows the diagram for this pin. The GP5 pin is configurable to function as one of the following:

- a general purpose I/O
- a TMR1 clock input
- a crystal/resonator connection
- a clock input

FIGURE 4-6: BLOCK DIAGRAM OF GP5



PIC12F683

TABLE 4-1: SUMMARY OF REGISTERS ASSOCIATED WITH GPIO

Addr	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOD	Value on all other Resets
05h	GPIO	—	—	GP5	GP4	GP3	GP2	GP1	GP0	--xx xx00	--uu uu00
0Bh/8Bh	INTCON	GIE	PEIE	T0IE	INTE	GPIE	T0IF	INTF	GPIF	0000 0000	0000 0000
19h	CMCON0	—	COUT	—	CINV	CIS	CM2	CM1	CM0	-0-0 0000	-0-0 0000
81h	OPTION_REG	GPPU	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0	1111 1111	1111 1111
85h	TRISIO	—	—	TRISIO5	TRISIO4	TRISIO3	TRISIO2	TRISIO1	TRISIO0	--11 1111	--11 1111
95h	WPU	—	—	WPU5	WPU4	—	WPU2	WPU1	WPU0	--11 -111	--11 -111
96h	IOC	—	—	IOC5	IOC4	IOC3	IOC2	IOC1	IOC0	--00 0000	--00 0000
9Fh	ANSEL	—	ADCS2	ADCS1	ADCS0	ANS3	ANS2	ANS1	ANS0	-000 1111	-000 1111

Legend: x = unknown, u = unchanged, — = unimplemented locations read as '0'. Shaded cells are not used by GPIO.